

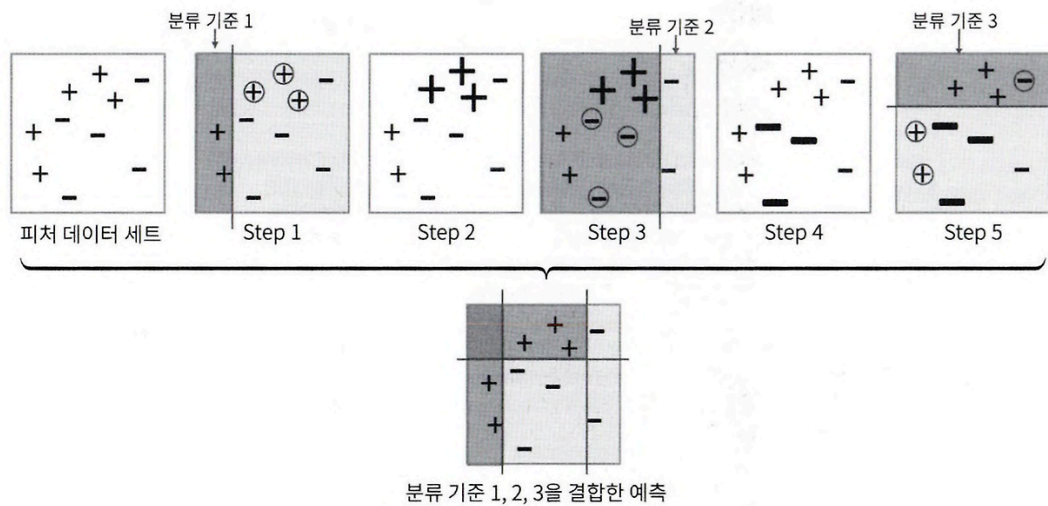
파머완

파머완 4장(분류) -Part2

05. GBM(Gradient Boosting Machine)

- 앙상블 학습의 **부스팅 알고리즘**: 여러개의 약한 학습기를 순차적으로 학습-예측하면서, 잘못 예측한 데이터에 가중치를 부여해 오류를 개선해 나가며 학습하는 방식. → AdaBoost, 그래디언트 부스트

1. AdaBoost(Adaptive Boosting)



- 앞의 학습기가 잘못 분류한 오류 데이터에 가중치 부여 → 다음 학습기가 더 잘 분류할 수 있도록 함.
- 개별 약한 학습기는 위 그림과 같이 각각 가중치 부여해 결합.

2. GBM

- 사이킷런의 **GradientBoostingClassifier** 클래스
- **GBM 기반 ML 패키지 : XGBoost, LightGBM**
- CART 기반 → 분류, 회귀 가능
- 과적합에 강함.
- **AdaBoost와 차이 : 가중치 업데이트 값 도출을 경사하강법으로 함.**
 - 경사하강법: 오류식 $h(x) = y - F(x)$ 을 최소화 할 수 있도록 가중치 값을 업데이트
- GBM vs 랜덤 포레스트
 - 예측성능 : GBM > 랜덤 포레스트
 - 수행 시간: GBM(병렬처리x) > 랜덤 포레스트 → 개선 필요.
 - GBM은 하이퍼 파라미터 튜닝 노력도 더 필요함

GBM 하이퍼 파라미터

- 트리처럼 **n_estimators, max_depth, max_features** 등 사용.
- 추가된 파라미터들

파라미터명	디폴트	설명
loss	'deviance'	경사하강법에서 사용할 비용 함수 지정

learning_rate(0~1)	0.1	학습을 진행할 때마다 적용하는 학습률. weak learner가 내놓는 보정값을 얼마나 반영할지. -작으면, 학습을 조금씩 함. (느리다) -크면, 학습을 한 번에 많이 함. (빠르지만, 예측 성능 떨어질 가능성 높다)
n_estimators	100	weak learner 개수 learning_rate를 적게 할 경우, n_estimators를 크게 한다. (상호보완적 사용)
subsample	1 (전체 학습데이터를 기반으로 학습)	학습에 사용하는 데이터의 샘플링 비율. 과적합이 염려되면, 1보다 작은 값.

06. XGBoost(eXtra Gradient Boost)

XGBoost 개요

- 장점

항목	설명
뛰어난 성능	분류, 회귀에서 예측 성능 ↑
GBM 대비 빠른 수행 시간	병렬 CPU 환경에서 병렬 학습 가능 → 수행시간 ↓
과적합 규제 (Regularization)	XGboost 자체 과적합 규제 기능으로 GBM 보다 좀 더 강한 내구성
Tree pruning(나무 가지치기)	GBM 은 분할 시 부정 손실 발행하면 더 이상 수행 X . but 이 경우도 과적합 발생 가능 → max_depth + tree pruning 으로 더 이상 긍정 이득 없는 분할을 가지치기 → 분할 수를 더 줄임
자체 내장된 교차 검증	반복 수행마다 내부적으로 교차 검증. 지정된 반복 횟수가 아니라, 평가값이 최적화되면 반복을 중간에 멈출 수 있는 조기 중단 기능 o
결손값 자체 처리	결손 값 자체 처리 기능 o

- 파이썬 래퍼 XGBoost: 고유의 API와 하이퍼 파라미터.
- 사이킷런 래퍼 XGBoost: fit, predict 같이 다른 Estimator와 사용법 같음.
- `import xgboost as xgb`

파이썬 래퍼 XGBoost 하이퍼 파라미터

- 파이썬 래퍼 XGBoost 모듈과 사이킷런 래퍼 XGBoost 모듈은 일부 하이퍼 파라미터가 다름 주의. (동일한 기능 but 이름 다름)
- 파이썬 래퍼 XGBoost 의 하이퍼 파라미터
 - 일반 파라미터 : 대부분 디폴트 값 사용
 - 부스터 파라미터 : 트리 최적화, 부스팅, regularization 등
 - 학습 태스크 파라미터 : 학습 수행 시 객체함수, 평가를 위한 지표 등 설정

• 주요 일반 파라미터

- **booster**: gbtree(tree based model) 또는 gblinear(linear model) 선택. 디폴트는 gbtree입니다.
- **silent**: 디폴트는 0이며, 출력 메시지를 나타내고 싶지 않을 경우 1로 설정합니다.
- **nthread**: CPU의 실행 스레드 개수를 조정하며, 디폴트는 CPU의 전체 스레드를 다 사용하는 것입니다. 멀티 코어/스레드 CPU 시스템에서 전체 CPU를 사용하지 않고 일부 CPU만 사용해 ML 애플리케이션을 구동하는 경우에 변경합니다.

• 부스터 파라미터

- `eta [default=0.3, alias: learning_rate]`: GBM의 학습률(learning rate)과 같은 파라미터입니다. 0에서 1 사이의 값을 지정하며 부스팅 스텝을 반복적으로 수행할 때 업데이트되는 학습률 값. 파이썬 래퍼 기반의 xgboost를 이용할 경우 디폴트는 0.3. 사이킷런 래퍼 클래스를 이용할 경우 `eta`는 `learning_rate` 파라미터로 대체되며, 디폴트는 0.1입니다. 보통은 0.01 ~ 0.2 사이의 값을 선호합니다.
- `num_boost_rounds`: GBM의 `n_estimators`와 같은 파라미터입니다.
- `min_child_weight[default=1]`: 트리에서 추가적으로 가지를 나눌지를 결정하기 위해 필요한 데이터들의 `weight` 총합. `min_child_weight`이 클수록 분할을 자제합니다. 과적합을 조절하기 위해 사용됩니다.
- `gamma [default=0, alias: min_split_loss]`: 트리의 리프 노드를 추가적으로 나눌지를 결정할 최소 손실 감소 값입니다. 해당 값보다 큰 손실(loss)이 감소된 경우에 리프 노드를 분리합니다. 값이 클수록 과적합 감소 효과가 있습니다.
- `max_depth[default=6]`: 트리 기반 알고리즘의 `max_depth`와 같습니다. 0을 지정하면 깊이에 제한이 없습니다. `Max_depth`가 높으면 특정 피쳐 조건에 특화되어 룰 조건이 만들어지므로 과적합 가능성이 높아지며 보통은 3~10 사이의 값을 적용합니다.
- `sub_sample[default=1]`: GBM의 `subsample`과 동일합니다. 트리가 커져서 과적합되는 것을 제어하기 위해 데이터를 샘플링하는 비율을 지정합니다. `sub_sample=0.5`로 지정하면 전체 데이터의 절반을 트리를 생성하는 데 사용합니다. 0에서 1 사이의 값이 가능하나 일반적으로 0.5 ~ 1 사이의 값을 사용합니다.
- `colsample_bytree[default=1]`: GBM의 `max_features`와 유사합니다. 트리 생성에 필요한 피쳐(칼럼)를 임의로 샘플링하는 데 사용됩니다. 매우 많은 피쳐가 있는 경우 과적합을 조정하는 데 적용합니다.
- `lambda [default=1, alias: reg_lambda]`: L2 Regularization 적용 값입니다. 피쳐 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있습니다.
- `alpha [default=0, alias: reg_alpha]`: L1 Regularization 적용 값입니다. 피쳐 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있습니다.
- `scale_pos_weight [default=1]`: 특정 값으로 치우친 비대칭한 클래스로 구성된 데이터 세트의 균형을 유지하기 위한 파라미터입니다.

- **학습 태스크 파라미터**

- **objective:** 최솟값을 가져야 할 손실 함수를 정의합니다. XGBoost는 많은 유형의 손실함수를 사용할 수 있습니다. 주로 사용되는 손실함수는 이진 분류인지 다중 분류인지에 따라 달라집니다.
- **binary:logistic:** 이진 분류일 때 적용합니다.
- **multi:softmax:** 다중 분류일 때 적용합니다. 손실함수가 multi:softmax일 경우에는 레이블 클래스의 개수인 num_class 파라미터를 지정해야 합니다.
- **multi:softprob:** multi:softmax와 유사하나 개별 레이블 클래스의 해당되는 예측 확률을 반환합니다.
- **eval_metric:** 검증에 사용되는 함수를 정의합니다. 기본값은 회귀인 경우는 rmse, 분류일 경우에는 error입니다. 다음은 eval_metric의 값 유형입니다.
 - rmse: Root Mean Square Error
 - mae: Mean Absolute Error
 - logloss: Negative log-likelihood
 - error: Binary classification error rate (0.5 threshold)
 - merror: Multiclass classification error rate
 - mlogloss: Multiclass logloss
 - auc: Area under the curve

• 과적합 문제가 심각할 경우 적용

- eta값 낮추기(0.01~0.1). 이 경우, num_round 또는 n_estimator 높여주기.
- max_depth 값 낮추기
- min_child_weight 값 높이기
- gamma 값 높이기
- subsample, colsample_bytree 조정

파이썬 래퍼 XGBoost 적용 - 위스콘신 유방암 예측

(실습 위주)

- XGBoost의 파이썬 패키지인 xgboost는 자체적으로 교차 검증, 성능 평가, 피쳐 중요도 등의 시각화 기능 O.
- **DMatrix 사용**
 - 넘파이, 판다스로 되어있는 학습용, 검증, 테스트용 데이터 세트를 모두 DMatrix로 생성하여 모델에 입력 필요.
 - `xgb.DMatrix(data=, label=)`
 - 주요 입력 파라미터: data(피쳐 데이터 세트), label(분류의 경우 레이블 데이터 세트, 회귀의 경우 숫자형인 종속값 데이터 세트)

- (학습) `xgb.train()`: 설정한 하이퍼파라미터와 학습 데이터를 받아서 XGBoost 모델을 학습시키고, 학습이 완료된 모델을 반환
 - `dtrain` : 학습 데이터 지정. (학습용 DMatrix 지정)
 - `num_boost_round` : 지정한 부스팅 반복횟수. (도달하지 않더라도, 더이상 지표 개선이 없을 경우 중지.)
 - `early_stopping_rounds` : 조기종단할 수 있는 최소 반복 횟수 설정.
 - `params` : XGBoost 모델의 하이퍼파라미터 설정
 - `evals= [(검증용 DMatrix, 'eval')]` : 조기종단을 위해 학습 중간에 성능을 확인하기 위한 옵션. 검증용 데이터를 넣어준다. → 학습 반복시마다 `evals` 에 설정된 데이터 세트에 대해 평가 지표 결과가 출력.
- (예측) 학습완료모델.`predict(테스트용 DMatrix)` : 사이킷런의 `predict`와 다르게 예측 결과값이 아닌 예측 결과를 추정할 수 있는 확률 반환. -> 예측 확률이 0.5보다 크면 1, 그렇지 않으면 0으로 예측 값을 결정하는 로직 추가하기
- (평가) 지금까지 했던 사이킷런의 방식과 같다.
- (시각화 도출) `plot_importance` 모듈 사용.

파이썬래퍼 XGBoost에서 교차검증 후, 최적 파라미터 구하기 - `cv()`

- 사이킷런의 `GridSearchCV`와 유사. 파이썬 래퍼 XGBoost에서 교차검증후, 최적 파라미터 구하는 api.
- 반환값은 DataFrame
- 파라미터
 - `params (dict)`: 부스터 파라미터
 - `dtrain (DMatrix)`: 학습 데이터
 - `num_boost_round (int)`: 부스팅 반복 횟수
 - `nfold (int)`: CV 폴드 개수.
 - `stratified (bool)`: CV 수행 시 층화 표본 추출(stratified sampling) 수행 여부
 - `metrics (string or list of strings)`: CV 수행 시 모니터링할 성능 평가 지표
 - `early_stopping_rounds (int)`: 조기 종단을 활성화시킴. 반복 횟수 지정.

사이킷런 래퍼 XGBoost의 개요 및 적용

- XGBClassifier, XGBRegressor 클래스가 있음.
- **import xgboost as xgb from xgboost import XGBClassifier**
- 변경된 하이퍼 파라미터
 - eta → learning_rate
 - sub_sample → subsample
 - lambda → reg_lambda (클수록 L2 규제 → 과적합 방지)
 - alpha → reg_alpha (클수록 L1 규제 → 과적합 방지)
 - num_boost_round, n_estimators는 동일한 파라미터. 동시 사용시, n_estimators 적용.
- plot_importance 모듈: 파이썬 래퍼 클래스와 똑같이 시각화 결과도출 가능. 각 feature가 XGBoost의 예측에 얼마나 중요했는지 나타냄.
 - **from xgboost import plot_importance**

조기종단 기능x

- 학습용, 테스트용으로만 분리
- XGBClassifier 설정
- fit, predict, predict_proba로 학습, 예측 수행

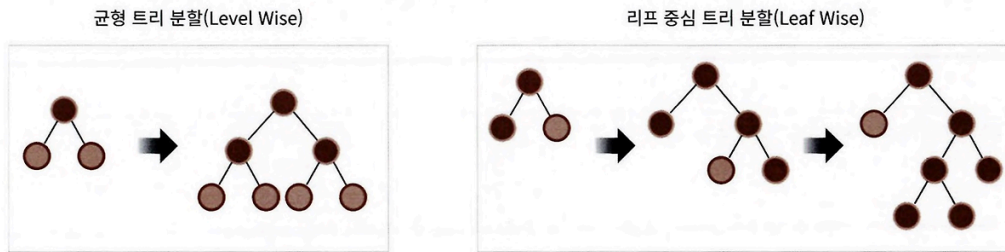
조기종단 기능 넣기 (early_stopping_rounds, eval_metric, eval_set)

- train_test_split으로 train + test 분리, train을 train_test_split으로 tr+val(검증용)으로 분리
- 조기종단 관련 파라미터들 - **early_stopping_rounds, eval_metric**을 XGBClassifier에 추가로 넣기.
- fit(X_tr, y_tr, **eval_set=**)으로 학습. 학습용, 검증용에 사용되는 실제 데이터셋 넣기.
 - * eval_set = [(X_tr, y_tr), (X_val, y_val)] 학습용/ 검증용으로 자동인식된다.
- predict, predict_proba로 예측 수행

07. LightGBM

LightGBM 개요

- 장점
 - GBM의 학습 시간을 개선한 XGBoost 보다도 학습시간 ↓
 - 메모리 사용량 상대적으로 ↓ but 성능은 XGBoost와 차이 없음.
 - **리프 중심 분할 방식** : 트리의 균형을 맞추지 않고, 최대 손실값(max delta loss)을 가지는 리프 노드를 지속적으로 분할하면서 트리의 깊이가 깊어지고 비대칭적인 규칙 트리 생성됨. → 균형 트리 방식(=일반 GBM 계열의 트리 분할 방법)보다 예측 오류 손실 최소화.



- 단점 : 적은 데이터 세트에 적용할 경우 과적합 발생이 쉬움

LightGBM 하이퍼 파라미터

- 주요 파라미터
 - `num_iterations` [default = 100]: 반복 수행하려는 트리의 개수를 지정합니다. 크게 지정할수록 예측 성능이 높아질 수 있으나, 너무 크게 지정하면 오히려 과적합으로 성능이 저하될 수 있습니다. 사이킷런 GBM과 XGBoost의 사이킷런 호환 클래스의 `n_estimators`와 같은 파라미터이므로 LightGBM의 사이킷런 호환 클래스에서는 `n_estimators`로 이름이 변경됩니다.
 - `learning_rate` [default = 0.1]: 0에서 1 사이의 값을 지정하며 부스팅 스텝을 반복적으로 수행할 때 업데이트되는 학습률 값입니다. 일반적으로 `n_estimators`를 크게 하고 `learning_rate`를 작게 해서 예측 성능을 향상시킬 수 있으나, 마찬가지로 과적합 이슈와 학습 시간이 길어지는 부정적인 영향도 고려해야 합니다. GBM, XGBoost의 `learning rate`와 같은 파라미터입니다.

- **max_depth** [default=-1]: 트리 기반 알고리즘의 max_depth와 같습니다. 0보다 작은 값을 지정하면 깊이에 제한이 없습니다. 지금까지 소개한 Depth wise 방식의 트리와 다르게 LightGBM은 Leaf wise 기반이므로 깊이가 상대적으로 더 깊습니다.
- **min_data_in_leaf** [default = 20]: 결정 트리의 min_samples_leaf와 같은 파라미터입니다. 하지만 사이킷런 래퍼 LightGBM 클래스인 LightGBMClassifier에서는 min_child_samples 파라미터로 이름이 변경됩니다. 최종 결정 클래스인 리프 노드가 되기 위해서 최소한으로 필요한 레코드 수이며, 과적합을 제어하기 위한 파라미터입니다.
- **num_leaves** [default = 31]: 하나의 트리가 가질 수 있는 최대 리프 개수입니다.
- **boosting** [default = gbdt]: 부스팅의 트리를 생성하는 알고리즘을 기술합니다.
 - gbdt: 일반적인 그래디언트 부스팅 결정 트리
 - rf: 랜덤 포레스트
- **bagging_fraction** [default = 1.0]: 트리가 커져서 과적합되는 것을 제어하기 위해서 데이터를 샘플링하는 비율을 지정합니다. 사이킷런의 GBM과 XGBClassifier의 subsample 파라미터와 동일하기에 사이킷런 래퍼 LightGBM인 LightGBMClassifier에서는 subsample로 동일하게 파라미터 이름이 변경됩니다.
- **feature_fraction** [default = 1.0]: 개별 트리를 학습할 때마다 무작위로 선택하는 피처의 비율입니다. 과적합을 막기 위해 사용됩니다. GBM의 max_features와 유사하며, XGBClassifier의 colsample_bytree와 똑같으므로 LightGBMClassifier에서는 동일하게 colsample_bytree로 변경됩니다.
- **lambda_l2** [default=0.0]: L2 regulation 제어를 위한 값입니다. 피처 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있습니다. XGBClassifier의 reg_lambda와 동일하므로 LightGBMClassifier에서는 reg_lambda로 변경됩니다.
- **lambda_l1** [default = 0.0]: L1 regulation 제어를 위한 값입니다. L2와 마찬가지로 과적합 제어를 위한 것이며, XGBClassifier의 reg_alpha와 동일하므로 LightGBMClassifier에서는 reg_alpha로 변경됩니다.

• Learning Task 파라미터

- **objective**: 최솟값을 가져야 할 손실함수를 정의합니다. Xgboost의 objective 파라미터와 동일합니다. 애플리케이션 유형, 즉 회귀, 다중 클래스 분류, 이진 분류인지에 따라서 objective인 손실함수가 지정됩니다.

하이퍼 파라미터 튜닝 방안

- **num_leaves** 의 개수를 중심으로 **min_child_samples**(min_data_in_leaf), **max_depth**를 함께 조정하면서 모델의 복잡도 줄이기
 - num_leaves(개별 트리가 가질 수 있는 최대 리프의 개수) 높이면, 정확도 but 트리 깊이가 깊어지고, 모델의 복잡도 과적합 영향
 - 사이킷런의 min_child_samples (= 파이썬 래퍼의 min_data_in_leaf) 높이면, 트리가 깊어지는 것 방지
 - max_depth: 깊이 크기를 직접 제한

- **learning_rate**를 작게 하면서 **n_estimators** 크게 하기 → **n_estimators**를 너무 크게 하면 과적합 발생 가능성을 유의
- **reg_lambda, reg_alpha** 같은 regularization 적용
- **conlsample_bytree, subsample** 적용 → 학습에 사용할 피쳐 개수나 데이터 샘플링 레코드 개수 줄이기

파이썬 래퍼 LightGBM과 사이킷런 래퍼 LightGBM, XGBoost 하이퍼 파라미터 비교

유형	파이썬 래퍼 LightGBM	사이킷런 래퍼 LightGBM	사이킷런 래퍼 XGBoost
파라미터명	num_iterations	n_estimators	n_estimators
	learning_rate	learning_rate	learning_rate
	max_depth	max_depth	max_depth
	min_data_in_leaf	min_child_samples	N/A
	bagging_fraction	subsample	subsample
	feature_fraction	colsample_bytree	colsample_bytree
	lambda_l2	reg_lambda	reg_lambda
	lambda_l1	reg_alpha	reg_alpha
	early_stopping_round	early_stopping_rounds	early_stopping_rounds
	num_leaves	num_leaves	N/A
	min_sum_hessian_in_leaf	min_child_weight	min_child_weight

LightGBM 적용 - 위스콘신 유방암 예측

- LGBMClassifier, LGBMRegressor 클래스가 있음
- **import lightgbm from lightgbm import LGBMClassifier**
- XGBoost와 동일하게 조기중단 가능.
- plot_importance 모듈: 파이썬 래퍼 클래스와 똑같이 시각화 결과도출 가능. 각 feature가 lightgbm의 예측에 얼마나 중요했는지 나타냄.
 - **from lightgbm import plot_importance**

조기중단 기능 넣기 (early_stopping_rounds, eval_metric, eval_set)

- train_test_split으로 train + test 분리, train을 train_test_split으로 tr+val(검증용)으로 분리

- 조기종단 관련 파라미터들 - **early_stopping_rounds**, **eval_metric**을 XGBClassifier에 추가로 넣기.
- fit(X_tr, y_tr, **eval_set=**)으로 학습. 학습용, 검증용에 사용되는 실제 데이터셋 넣기.
 - * eval_set = [(X_tr, y_tr), (X_val, y_val)] 학습용/ 검증용으로 자동인식된다.
- predict, predict_proba로 예측 수행

08. 베이지안 최적화 기반의 HyperOpt를 이용한 하이퍼 파라미터 튜닝

- Grid Search를 이용한 하이퍼파라미터 튜닝의 단점 : 튜닝해야할 파라미터 수 많은 경우 최적화 수행 시간↑
- XGBoost, LightGBM은 하이퍼 파라미터 개수가 매우 많다. → Grid Search를 이용하면 많은 시간 소모..

베이지안 최적화 개요

- 목적 함수 식을 제대로 알 수 없는 블랙박스 형태의 함수에서 최대 또는 최소 함수 반환 값을 만드는 최적 입력값을 가능한 적은 시도를 통해 빠르고 효과적으로 찾아주는 방식.
- 베이지안 확률 기반.
- 구성 요소
 - **대체 모델** : 획득 함수가 계산한 하이퍼 파라미터를 입력받음. 이를 기반으로 최적 함수 모델 개선. (최적 함수를 찾음)
 - **획득 함수** : 개선된 대체 모델 기반으로 최적 입력값 계산.

HyperOpt 사용하기

- 베이지안 최적화를 머신러닝 모델의 하이퍼 파라미터 튜닝에 적용할 수 있게 제공되는 패키지.
- 목적 함수 **반환 값의 최솟값**을 가지는 최적입력값을 유추
 - 목적 함수: 최종적으로 좋게 만들고 싶은것. 최적화하려는 대상. 평가하고싶은 값.
ex. 정확도

- 1.입력변수명, 입력값의 검색공간 설정 2.목적 함수 설정 3.목적 함수의 반환 최소값을 가지는 최적 입력값 유추

1. 입력 변수명, 입력값의 검색공간 설정

- hp 모듈 이용. `from hyperopt import hp`
 - 딕셔너리 형태로 검색공간 설정: **key는 입력 변수명, value는 검색공간**
 - ex. `search_space={'x':hp.quniform('x', -10, 10, 1) , 'y':hp.quniform('y', -15, 15, 1)}`
 - 검색공간을 제공하는 대표적인 함수들
 - **label: 입력 변수명, low:최솟값, high: 최댓값, q: 간격.**
- `hp.quniform(label, low, high, q)`: label로 지정된 입력값 변수 검색 공간을 최솟값 low에서 최댓값 high까지 q의 간격을 가지고 설정.
 - `hp.uniform(label, low, high)`: 최솟값 low에서 최댓값 high까지 정규 분포 형태의 검색 공간 설정
 - `hp.randint(label, upper)`: 0부터 최댓값 upper까지 random한 정숫값으로 검색 공간 설정.
 - `hp.loguniform(label, low, high)`: `exp(uniform(low, high))`값을 반환하며, 반환 값의 log 변환 된 값은 정규 분포 형태를 가지는 검색 공간 설정.
 - `hp.choice(label, options)`: 검색 값이 문자열 또는 문자열과 숫자값이 섞여 있을 경우 설정. Options는 리스트나 튜플 형태로 제공되며 `hp.choice('tree_criterion', ['gini', 'entropy'])`와 같이 설정하면 입력 변수 `tree_criterion`의 값을 'gini'와 'entropy'로 설정하여 입력함.

2. 목적함수 설정

- `from hyperopt import STATUS_OK`
- 인자(입력): 변수값과 검색공간을 가지는 딕셔너리
 - `def objective_func(search_space)`
- 반환값: 단일값, 딕셔너리 형태 가능
 - `return retval`
 - 딕셔너리인 경우, `{'loss': retval, 'status': STATUS_OK}`와 같이, `loss`, `status`값을 설정해서 반환

3. 목적 함수의 반환 최솟값을 가지는 최적입력값 유추

- `fmin(objective, space, algo, max_evals, trials)` 함수
- `from hyperopt import fmin, tpe, Trials`
- `trials = Trials()` <- Trials 객체로 넣어준다.
- `fmin()` 반환값: {'하이퍼파라미터1': 최적의 파라미터값, '하이퍼파라미터2': 최적의 파라미터값, ...}

- `fn`: 위에서 생성한 `objective_func`와 같은 목적 함수입니다.
- `space`: 위에서 생성한 `search_space`와 같은 검색 공간 딕셔너리입니다.
- `algo`: 베이지안 최적화 적용 알고리즘 입니다. 기본적으로 `tpe.suggest`이며 이는 HyperOpt의 기본 최적화 알고리즘인 TPE(Tree of Parzen Estimator)를 의미합니다.
- `max_evals`: 최적 입력값을 찾기 위한 입력값 시도 횟수입니다.
- `trials`: 최적 입력값을 찾기 위해 시도한 입력값 및 해당 입력값의 목적 함수 반환값 결과를 저장하는 데 사용됩니다. Trials 클래스를 객체로 생성한 변수명을 입력합니다.
- `rstate`: `fmin()`을 수행할 때마다 동일한 결괏값을 가질 수 있도록 설정하는 랜덤 시드(seed) 값입니다.

Trials 객체의 중요 속성 - results, vals

- `.results` : 함수 반복 수행시마다 반환되는 변환 값, 리스트 형태
 - `[{'loss': -64.0, 'status': 'ok'}, {'loss': -184.0, 'status': 'ok'}, ...]` → `max_evals` 만큼 원소(딕셔너리)가 리스트 안에 존재.
- `.vals` : 함수 반복 수행시마다 입력되는 입력 변수값, 딕셔너리 형태 ('입력변수명': 개별 수행시마다 입력된값 리스트)
 - `{'x': [-6.0, -4.0, 2.0, ..], 'y': [5.0, 10.0, -2.0, ..]}`
- `x, y, 반환값(loss)`을 피처로 가지는 Dataframe 만들기

```
losses= [loss_dict['loss'] for loss_dict in trial_val.results]
result_df= pd.DataFrame({'x':trial_val.vals['x'],
'y':trial_val.vals['y'], 'losses':losses})
#모든 배열들이 리스트형태.
```

HyperOpt를 이용한 XGBoost 하이퍼 파라미터 최적화

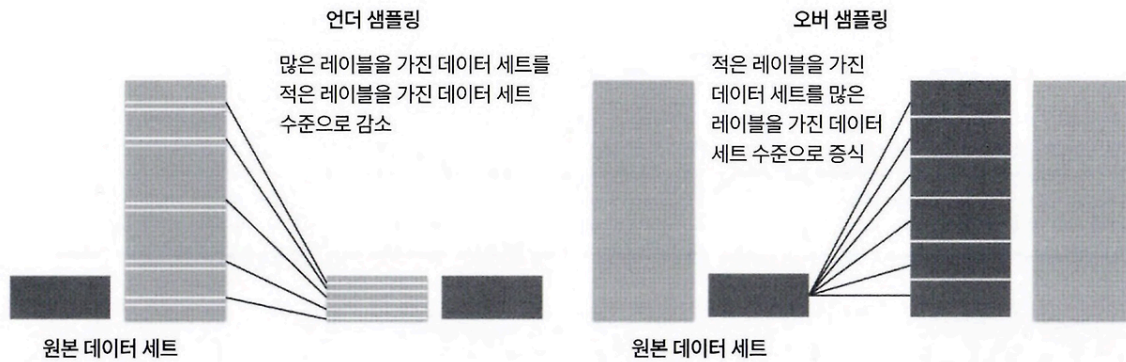
- 적용할 파라미터와 검색 공간 설정 - 목적 함수에서 XGBoost 학습 - 예측 성능 결과를 반환값 설정 - fmin() 함수
- 주의할점
 - HyperOpt의 입력값, 반환값이 모두 실수형 but 특정 하이퍼 파라미터는 정숫값만 입력 받는 경우. → 목적함수 만들 때, 해당 파라미터는 int 형변환 필요.
 - HyperOpt의 목적 함수는 최솟값을 반환해야 함. → **성능 값이 클수록 좋은 지표일 경우 -1 곱하기**
- 과정
 - 하이퍼 파라미터 검색 공간 설정
 - 목적 함수 만들기 - Classifier 모델 설정 , 반환값은 성능측정 지표(-1*정확도)
 - 함수로 최적 하이퍼 파라미터 도출
 - 도출된 최적의 하이퍼 파라미터를 Classifier에 설정 후 학습(fit) -> 성능 평가(predict, predict_proba)

10. 분류 실습 - 캐글 신용카드 사기 검출

- 데이터: 0(정상),1(사기) 로 분류. 1이 매우적음 - 레이블 값이 극도로 불균형한 분포
 - LGBClassifier 객체 생성시, boost_from_average= False

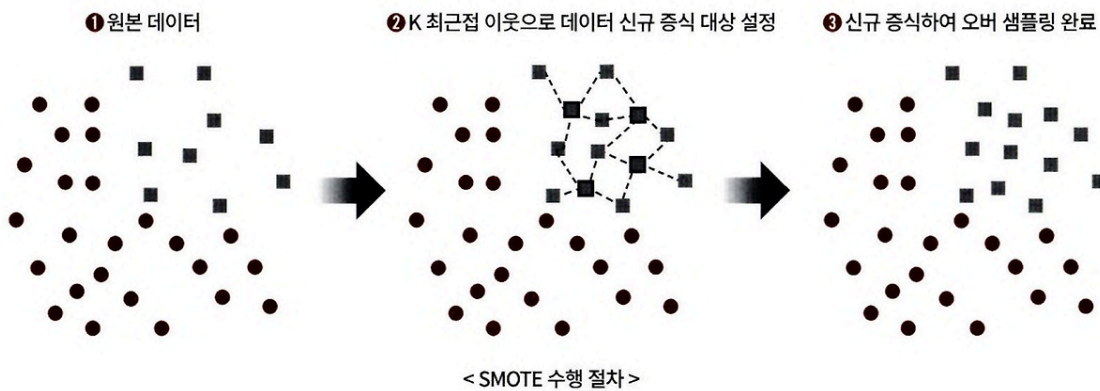
오버 샘플링과 언더 샘플링

- 지도 학습에서 극도로 불균형한 레이블 값 분포를 해결하는 방안.
- 오버 샘플링이 언더 샘플링보다 더 많이 사용됨
- 오버 샘플링 : 원본 데이터의 피쳐 값들을 조금 변경 후, 증식하는 방법.



오버 샘플링 - SMOTE 방법

: 적은 데이터 세트에 있는 개별 데이터들의 K 최근접 이웃(K Nearest Neighbor)을 찾아서 이 데이터와 K개 이웃들의 차이를 일정 값으로 만들어서 기존 데이터와 약간 차이가 나는 새로운 데이터를 생성



- 반드시 학습용 데이터세트만 오버샘플링.
- SMOTE를 적용한 학습용 데이터로 학습/예측/평가 → 재현율↑, 정밀도↓이 일반적.

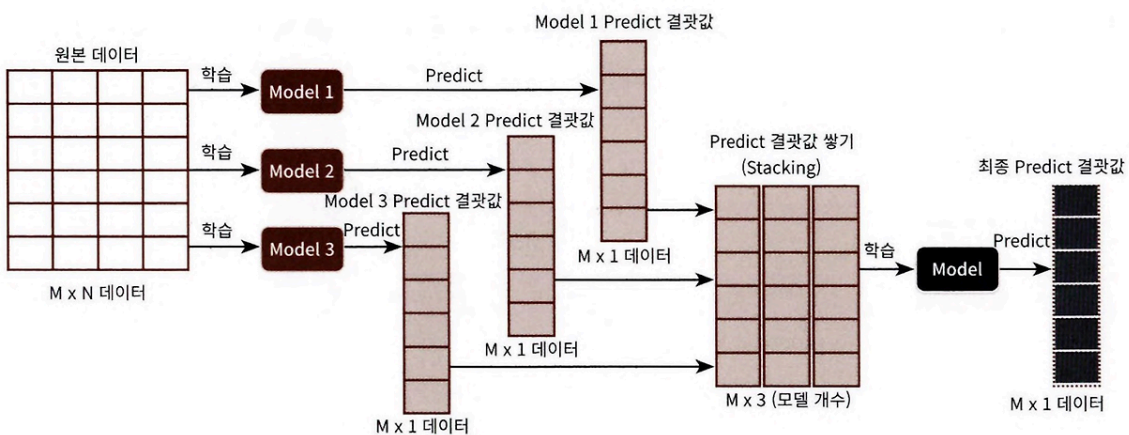
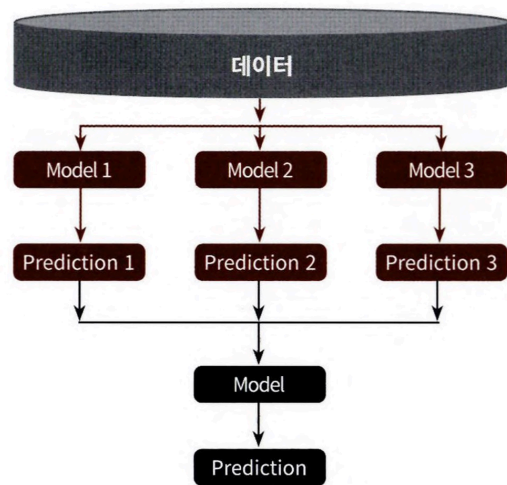
코드

- `from imblearn.over_sampling import SMOTE`
- `객체 = SMOTE()`
- `SMOTE 적용 후 학습용 X, SMOTE 적용 후 학습용 y = 객체.fit_resample(X_train, y_train)`

11. 스택킹 앙상블

스태킹 개요

- 여러 모델의 예측 결과를 새로운 모델이 다시 학습.
- 즉, 개별 알고리즘의 예측 결과 데이터 세트를 최종적인 메타 데이터 세트로 만들어 별도의 ML 알고리즘으로 최종학습을 수행하고, 테스트 데이터를 기반으로 다시 최종예측을 수행.
- 성능이 비슷한 모델을 결합 → 좀 더 나은 성능 향상을 도출하기 위해 사용됨.
- 구성
 - 개별적인 기반 모델
 - 개별 기반 모델의 예측 데이터를 학습 데이터로 만들어서 학습하는 최종 메타 모델



기본 스택킹 모델

- 개별 모델의 예측 데이터(predict한 값)를 스택킹으로 재구성해 최종 메타 모델에서 학습하고 예측.
 - 개별 ML 모델 생성

- 최종 메타 모델 생성
- 개별 모델들을 각각 학습(fit)후, 예측 데이터 세트 반환(predict의 반환값, ndarray)
- 예측 데이터 세트를 붙이고, 옆으로 눕혀(transpose) 스택킹 형태로 만듦 → 각 모델의 값을 각각 피쳐값으로 만들기
- 최종 메타 모델 학습/예측/정확도 측정

CV 세트 기반의 스택킹

- 과적합을 개선하기 위해, 최종 메타 모델을 위한 데이터 세트를 만들 때 교차 검증 기반으로 예측된 결과 데이터 세트 이용.
- 개별 모델들이 각각 교차 검증으로 메타 모델을 위한 학습용 스택킹 데이터, 테스트용 스택킹 데이터를 생성 → 메타 모델이 학습, 예측 수행

과정

1. 개별 모델들이 메타 모델을 위한 최종 학습용/테스트용 데이터 생성

KFold 교차검증 수행

먼저 전체 학습 데이터(X_{train} , y_{train})를 3개의 폴드로 나눈 뒤, 매 반복마다 2개 폴드는 기반 모델의 학습용, 나머지 1개 폴드는 검증용으로 사용함.

- 2개 폴드 → 학습(fit)
- 1개 폴드 → 검증 및 예측(predict)

이 과정을 3번 반복하여 모든 학습 데이터가 한 번씩 검증용으로 사용되도록 함.

- 검증 폴드 1개로 예측한 결과를 각 폴드마다 저장 → 3번 반복하여 X_{train} 전체에 대한 OOF 예측값 생성 → **메타 모델의 최종 학습 데이터**
- 각 fold에서 학습된 기반 모델로 전체 X_{test} 를 예측(predict) → fold별 예측값을 저장하고 평균 → **메타 모델의 최종 테스트 데이터**

2. 최종 학습용 데이터 + 원본 데이터의 y_{train} 으로 메타 모델 학습

3. 최종 테스트 데이터로 예측

4. 원본 테스트 데이터의 레이블 데이터 y_{test} 와 비교해 평가.